

Data Science & Simulations Part I

Camilo Aponte-Santamaría

BioMolecular Mechanics Dept. (Prof. Frauke Gräter)

April 29, 2026

Data Science & Simulations Part I:

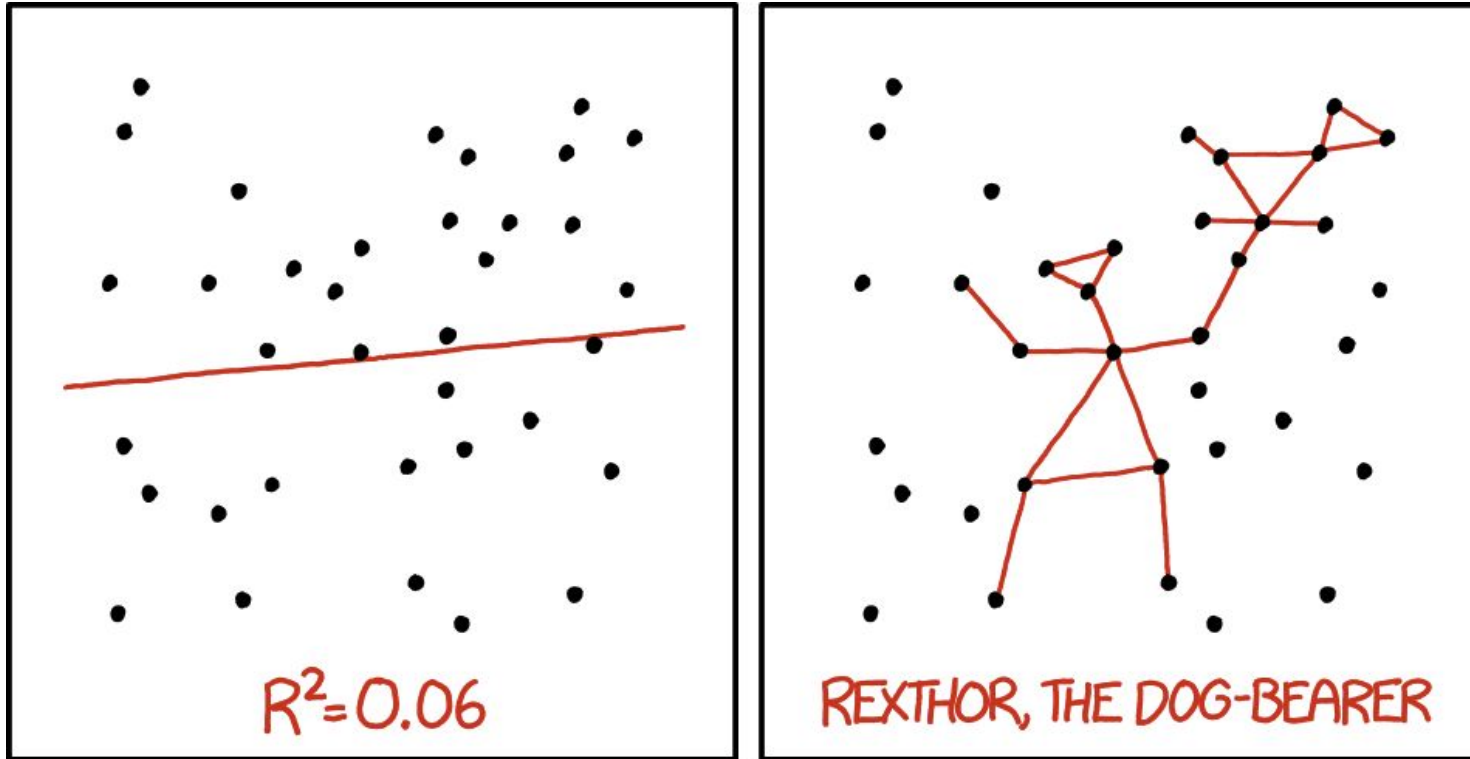
Goals:

Data Science	1.	Introduce the basic concepts of machine learning and related tools
	2.	Illustrate ML applications in the study of molecular systems
Simulations	3.	Introduce the method of molecular dynamics and its application to the study of biomolecular systems

Program:

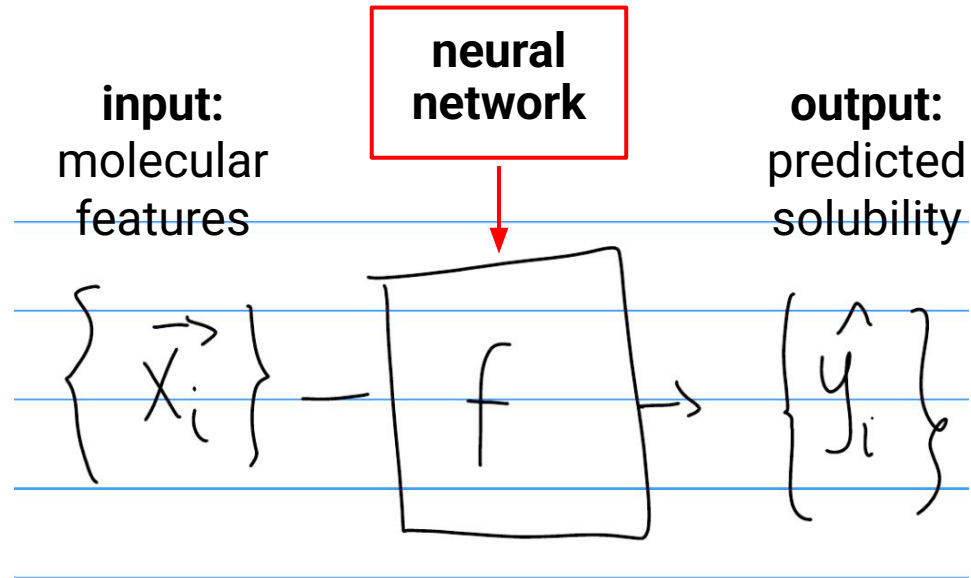
Date	Topic	L: Lecture T: Tutorial
15/04	Introduction to data science, data analysis, and machine learning	L
22/04	Introduction to data science, data analysis, and machine learning	T
29/04	Introduction to deep learning	L
06/05	Introduction to deep learning	T
13/05	Biomolecular dynamics simulations	L
20/05	Biomolecular dynamics simulations	T

why deep learning?
linear models are good, but...



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

what is deep learning?



- we use “**neural networks**” to do machine learning
- **deep**: many interconnected layers resembling neuronal connections

what is a neural network?

Linear model:

$$\hat{y} = \vec{w} \cdot \vec{x} + b$$

Model equation for deep-learning regression:

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

nonlinear function:

- operates on the input features: $\vec{x}$
- outputs new features: $\vec{h}$
- differentiable function
- composed of layers

Example: one “dense” layer

$$\vec{h} = g(\vec{x}) = \sigma[\vec{w} \cdot \vec{x} + \vec{b}]$$

Activation function:

- nonlinear
- typically step-wise (on/off parts of the data)

Weight matrix:

- $F \times D$ dimension
- F :
 - hyperparameter
 - layer width

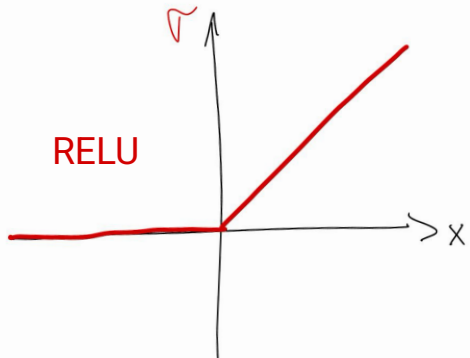
Feature vector:

- D -dimensional

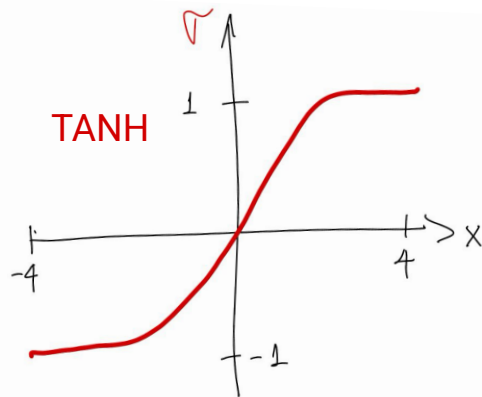
Bias vector:

- F -dimensional

ML task: find these parameters



$$\sigma(x) = \begin{cases} x, & x > 0 \\ 0 & \text{otherwise} \end{cases}$$



$$\hat{y} = \vec{w} \cdot \vec{h} + b$$
$$\vec{h} = g(\vec{x})$$

Example: one “dense” layer, $D=2$ features, $F=3$ width

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

$$\vec{h} = \sigma [\vec{w}^0 \vec{x} + \vec{b}^0]$$

Example: one “dense” layer, $D=2$ features, $F=3$ width

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

$$\vec{h} = \sigma[\vec{w}^0 \vec{x} + \vec{b}^0]$$

$$\hat{y} = [w_1 \ w_2 \ w_3] \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} + b$$

$$\vec{h} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \sigma \left[\begin{bmatrix} w_{11}^0 & w_{12}^0 \\ w_{21}^0 & w_{22}^0 \\ w_{31}^0 & w_{32}^0 \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1^0 \\ b_2^0 \\ b_3^0 \end{bmatrix} \right]$$

Example: one “dense” layer, $D=2$ features, $F=3$ width
writing the equations explicitly:

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

$$\vec{h} = \sigma[\vec{w}^0 \vec{x} + \vec{b}^0]$$

$$\hat{y} = \sum_{i=1}^3 w_i h_i + b$$

$$h_1 = \sigma[w_{11}^0 \cdot x_1 + w_{12}^0 \cdot x_2 + b_1^0]$$

$$h_2 = \sigma[w_{21}^0 \cdot x_1 + w_{22}^0 \cdot x_2 + b_2^0]$$

$$h_3 = \sigma[w_{31}^0 \cdot x_1 + w_{32}^0 \cdot x_2 + b_3^0]$$

Example: one “dense” layer, $D=2$ features, $F=3$ width
graphical (network-like) representation:

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

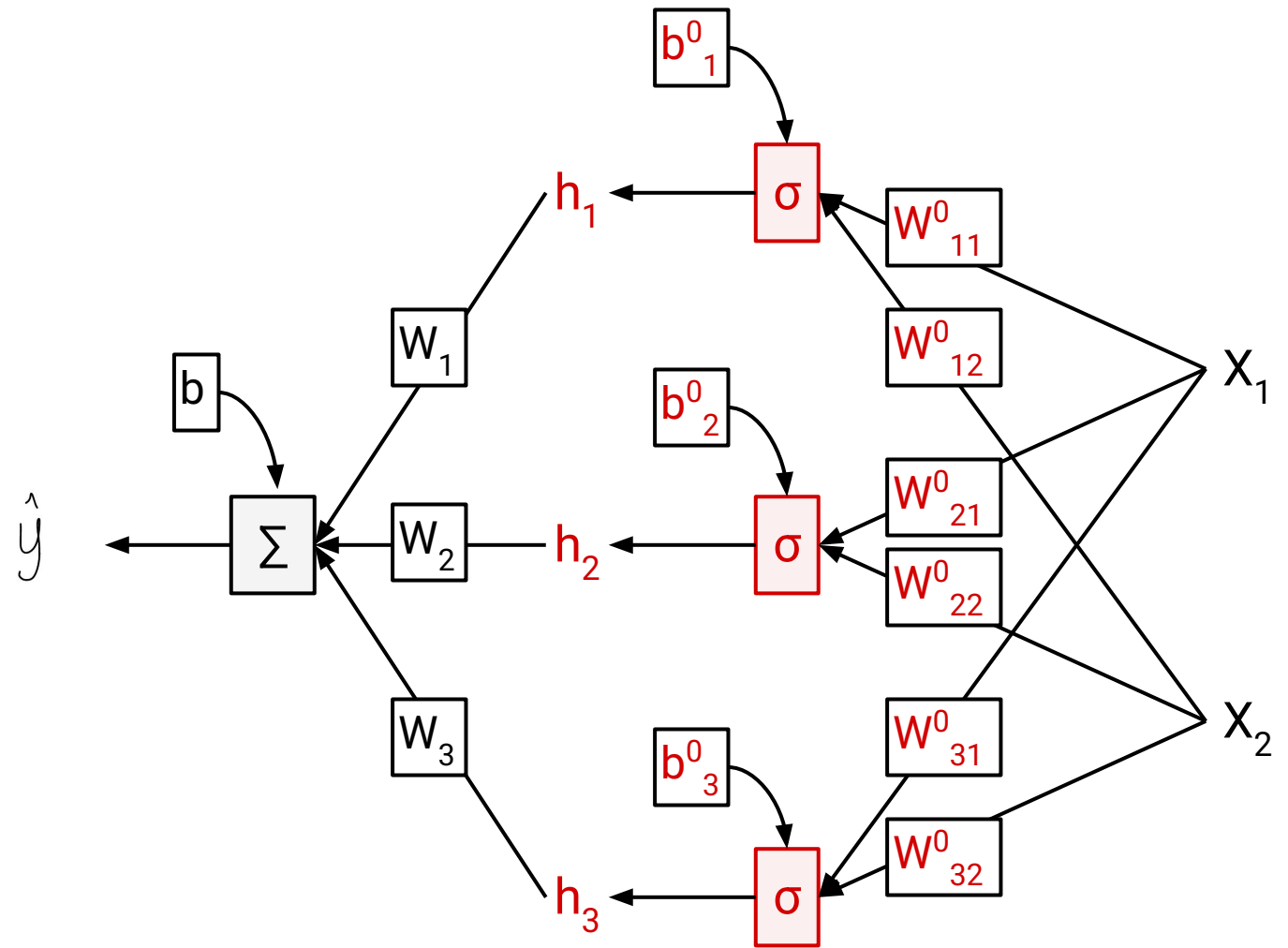
$$\vec{h} = \sigma[\vec{w}^0 \vec{x} + \vec{b}^0]$$

$$\hat{y} = \sum_{i=1}^3 w_i h_i + b$$

$$h_1 = \sigma[w_{11}^0 \cdot x_1 + w_{12}^0 \cdot x_2 + b_1^0]$$

$$h_2 = \sigma[w_{21}^0 \cdot x_1 + w_{22}^0 \cdot x_2 + b_2^0]$$

$$h_3 = \sigma[w_{31}^0 \cdot x_1 + w_{32}^0 \cdot x_2 + b_3^0]$$



Example: one “dense” layer, $D=2$ features, $F=3$ width
graphical (network-like) representation:

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

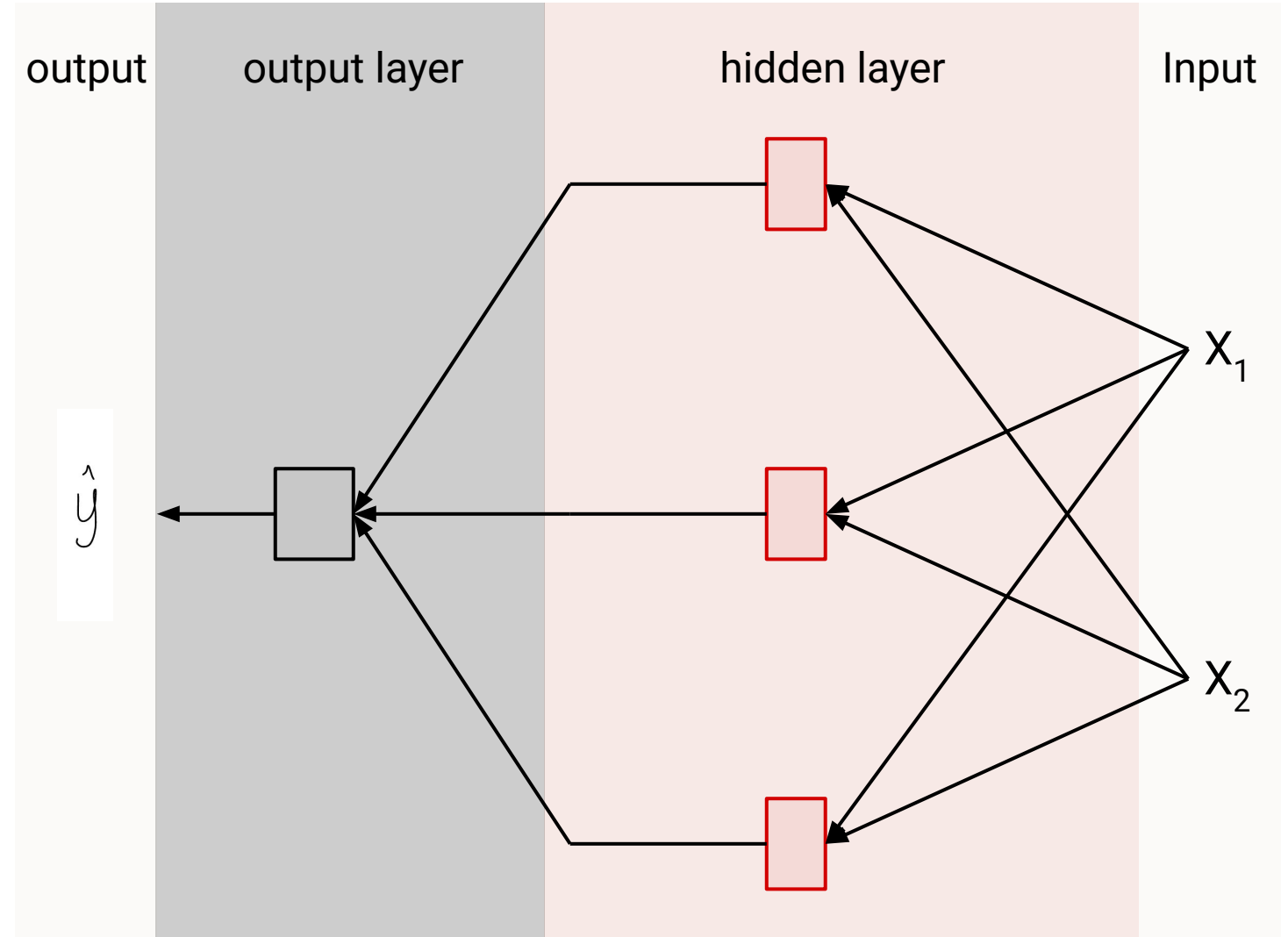
$$\vec{h} = \sigma[\vec{w}^0 \vec{x} + \vec{b}^0]$$

$$\hat{y} = \sum_{i=1}^3 w_i h_i + b$$

$$h_1 = \sigma[w_{11}^0 \cdot x_1 + w_{12}^0 \cdot x_2 + b_1^0]$$

$$h_2 = \sigma[w_{21}^0 \cdot x_1 + w_{22}^0 \cdot x_2 + b_2^0]$$

$$h_3 = \sigma[w_{31}^0 \cdot x_1 + w_{32}^0 \cdot x_2 + b_3^0]$$



Example: one “dense” layer, $D=2$ features, $F=3$ width graphical (network-like) representation (flipped):

$$\hat{y} = \vec{w} \cdot \vec{h} + b$$

$$\vec{h} = g(\vec{x})$$

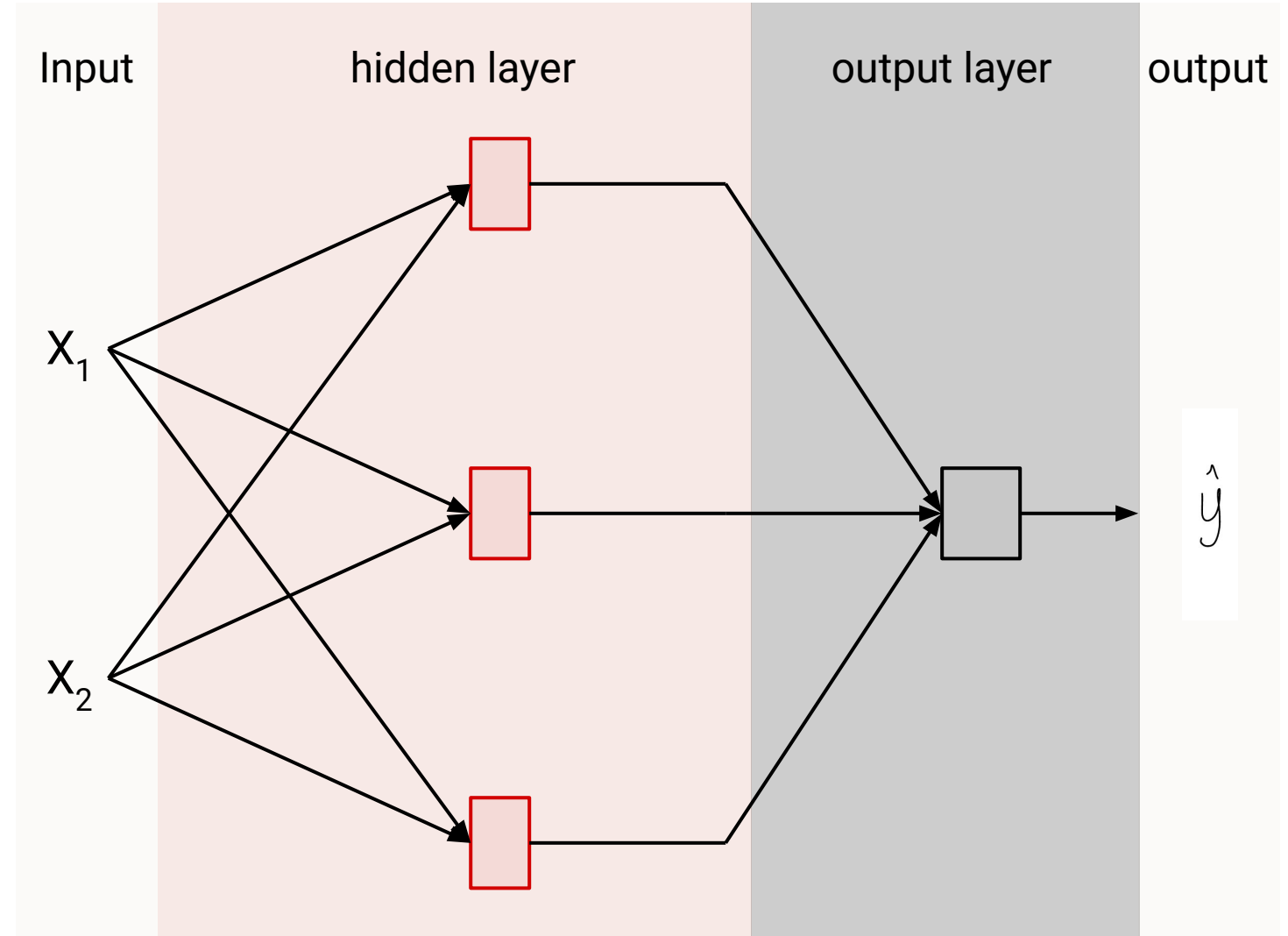
$$\vec{h} = \sigma[\vec{w}^0 \vec{x} + \vec{b}^0]$$

$$\hat{y} = \sum_{i=1}^3 w_i h_i + b$$

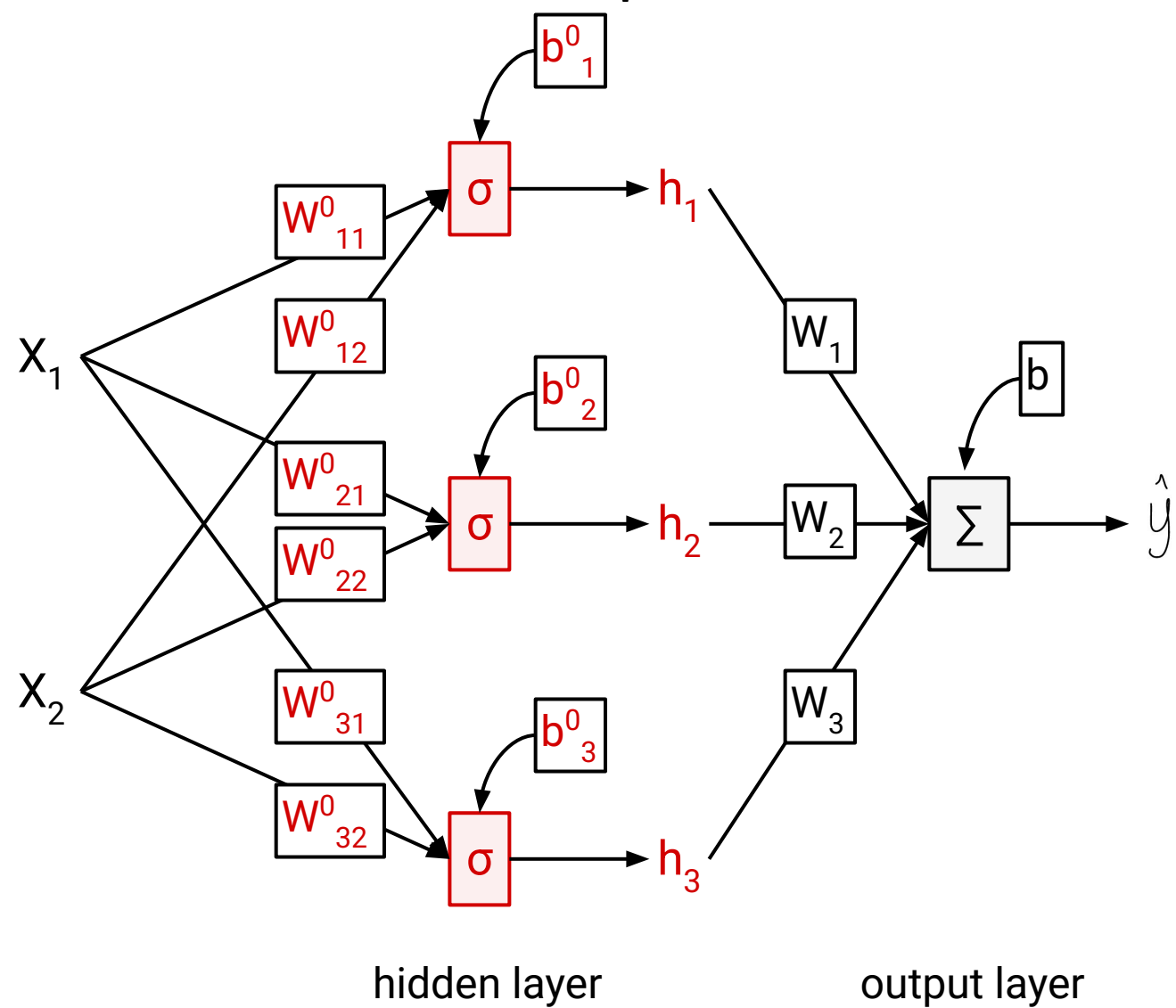
$$h_1 = \sigma[w_{11}^0 \cdot x_1 + w_{12}^0 \cdot x_2 + b_1^0]$$

$$h_2 = \sigma[w_{21}^0 \cdot x_1 + w_{22}^0 \cdot x_2 + b_2^0]$$

$$h_3 = \sigma[w_{31}^0 \cdot x_1 + w_{32}^0 \cdot x_2 + b_3^0]$$



Number of free parameters



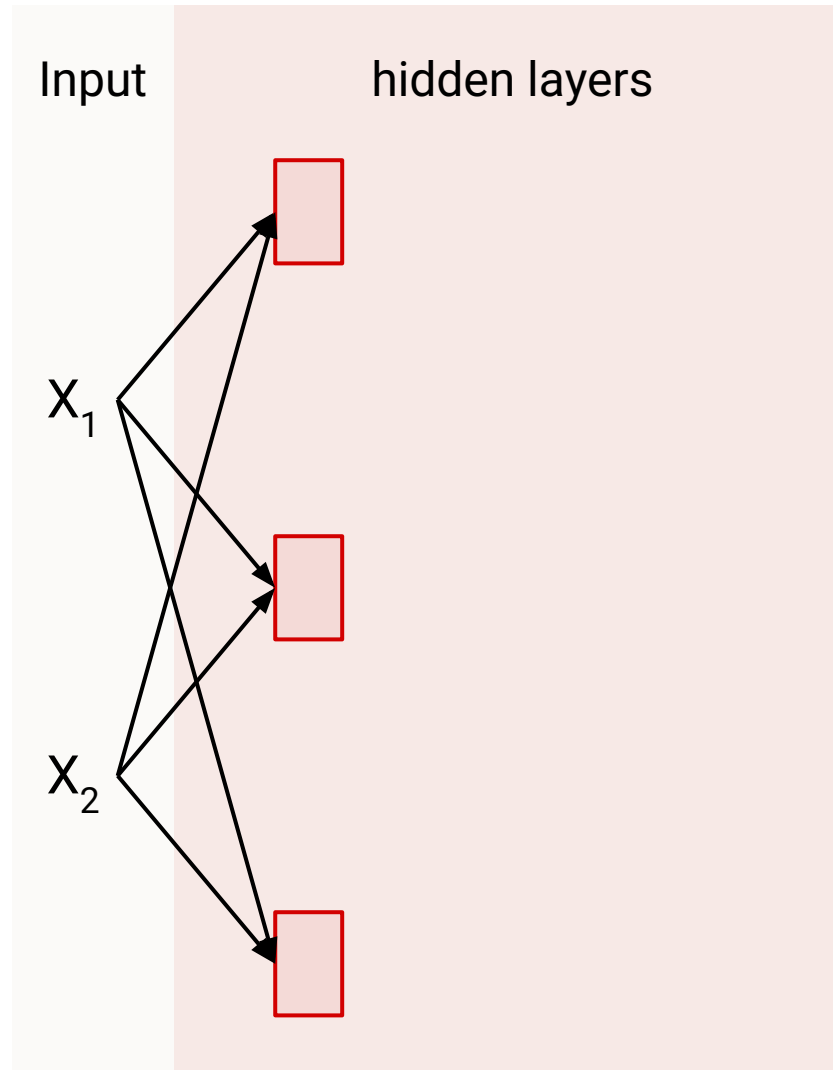
		2x3 example	one layer: D features F width ?
hidden layer	weight matrix W^0	6	$F \times D$
	bias b^0	3	F
output layer	weight matrix W	3	F
	bias b	1	1
Total		13	$F(D+2)+1$

More “dense” layers

$$\sigma \left[\vec{w}^o \cdot \vec{x} + b^o \right]$$

↓

$$\vec{h}$$



More “dense” layers

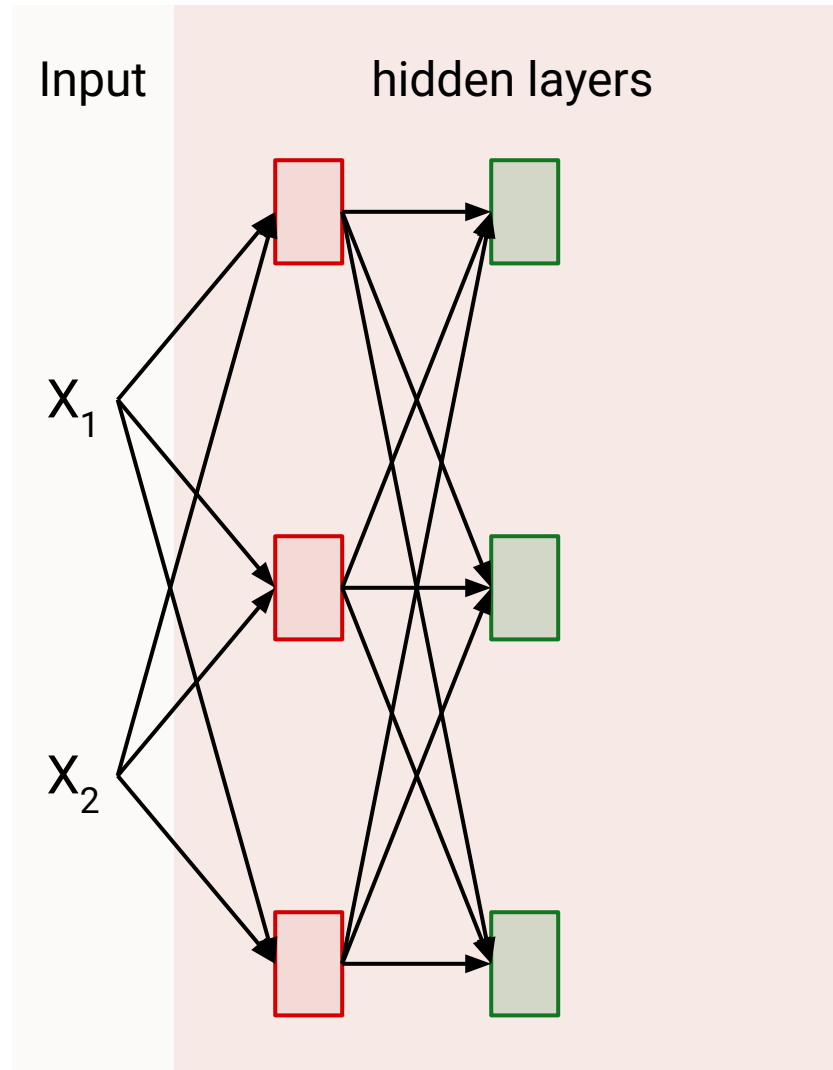
$$\sigma \left[\vec{w}^0 \cdot \vec{x} + b^0 \right]$$

↓

$$\sigma \left[\vec{w}^1 \cdot \vec{h} + b^1 \right]$$

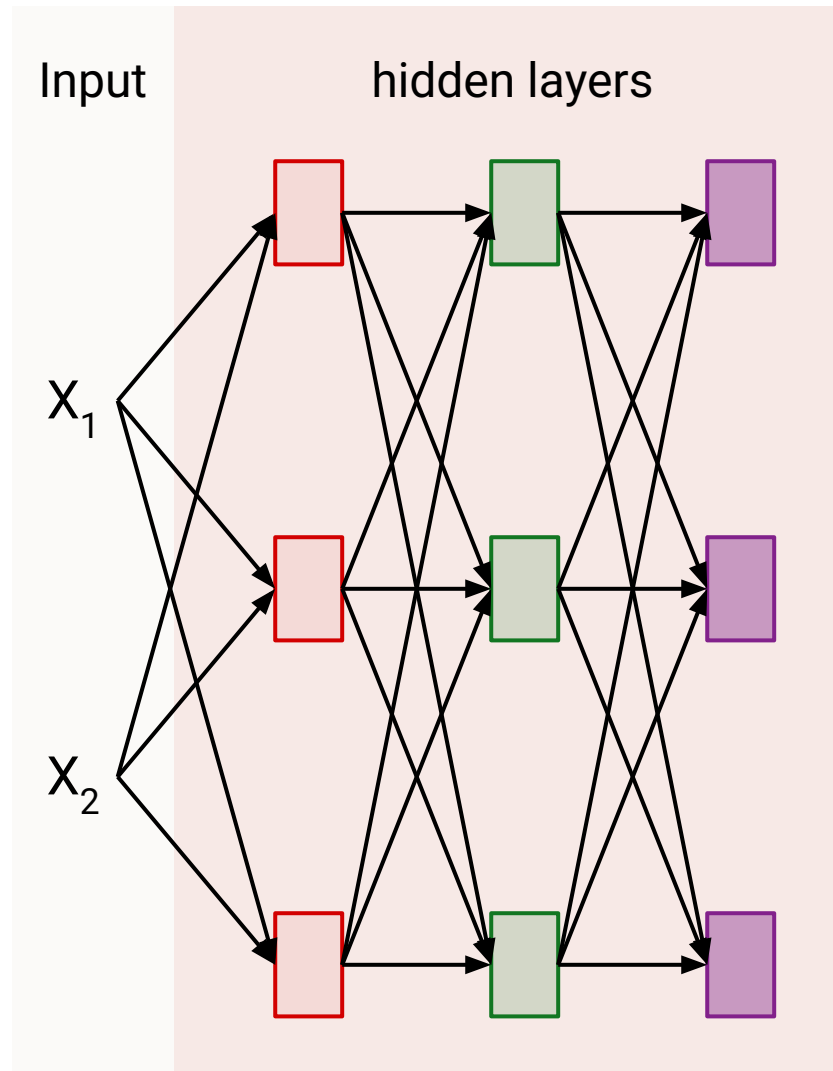
↓

$$\vec{h}^*$$



More “dense” layers

$$\underbrace{\sigma[\vec{w}^0 \cdot \vec{x} + \vec{b}^0]}_{\downarrow}$$
$$\underbrace{\sigma[\vec{w}^1 \cdot \vec{h} + \vec{b}^1]}_{\downarrow}$$
$$\underbrace{\sigma[\vec{w}^2 \cdot \vec{h}^* + \vec{b}^2]}_{\downarrow}$$
$$\vec{h}^{**}$$



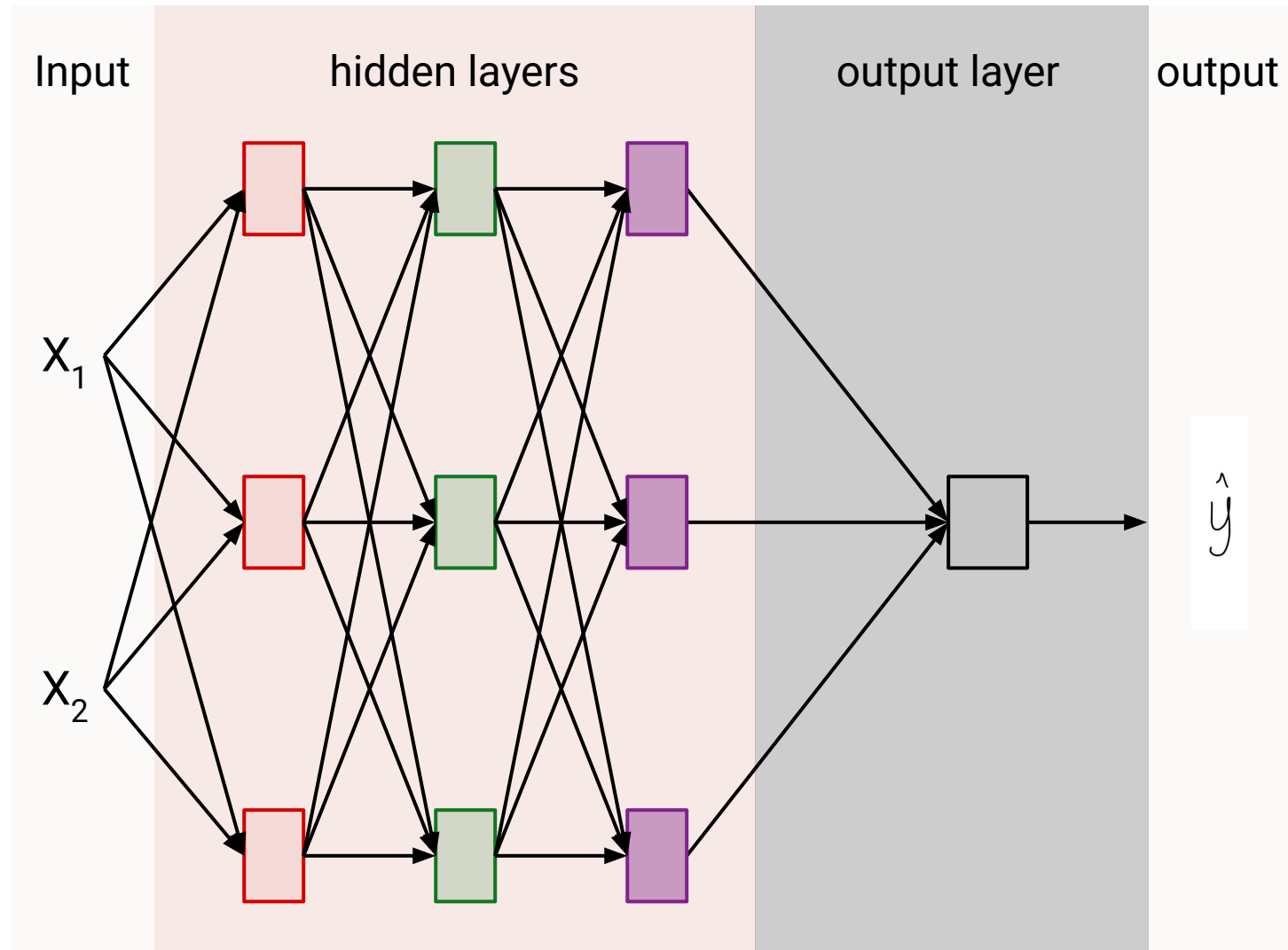
More “dense” layers

$$\underbrace{\sigma[\vec{w}^0 \cdot \vec{x} + b^0]}_{\downarrow}$$

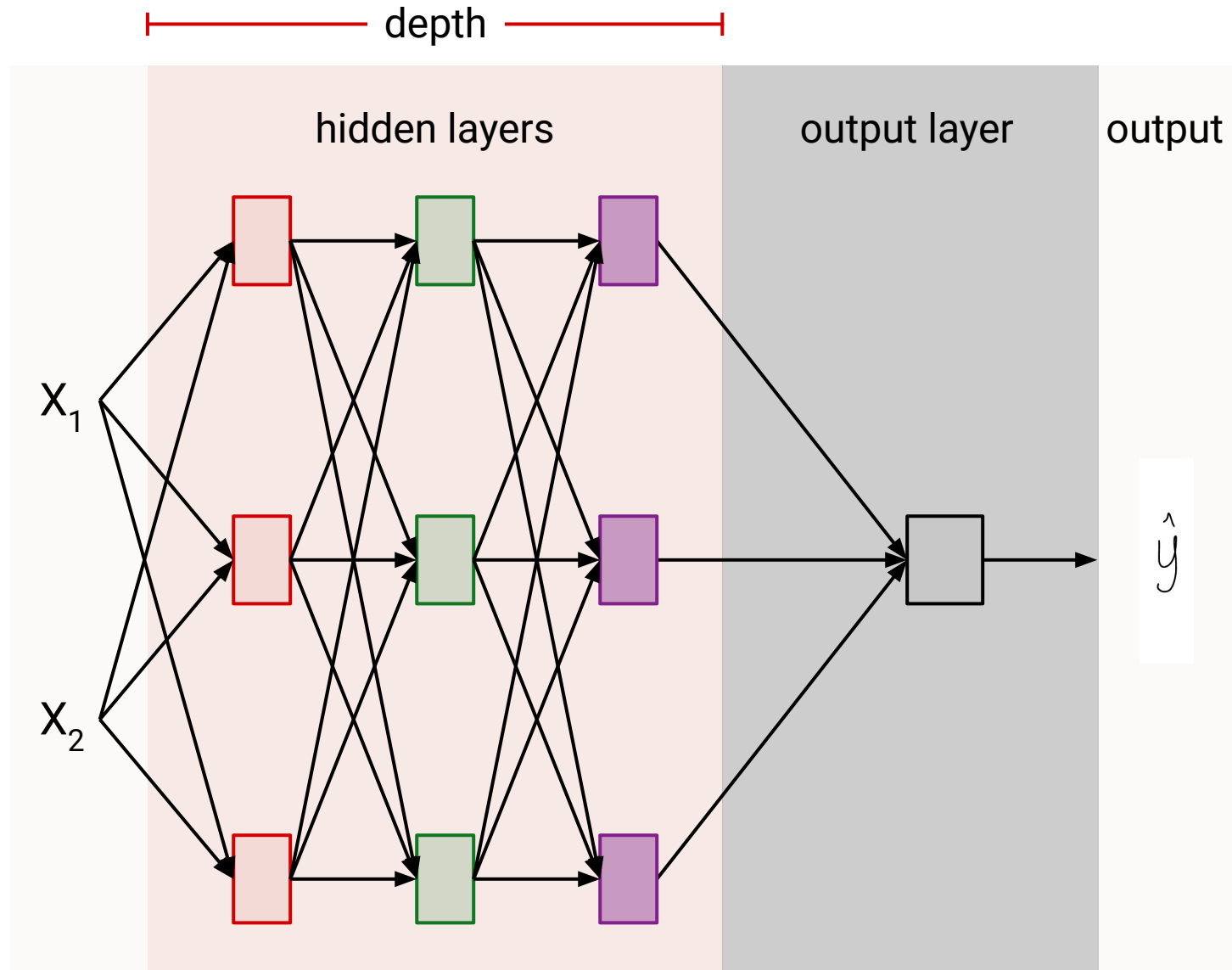
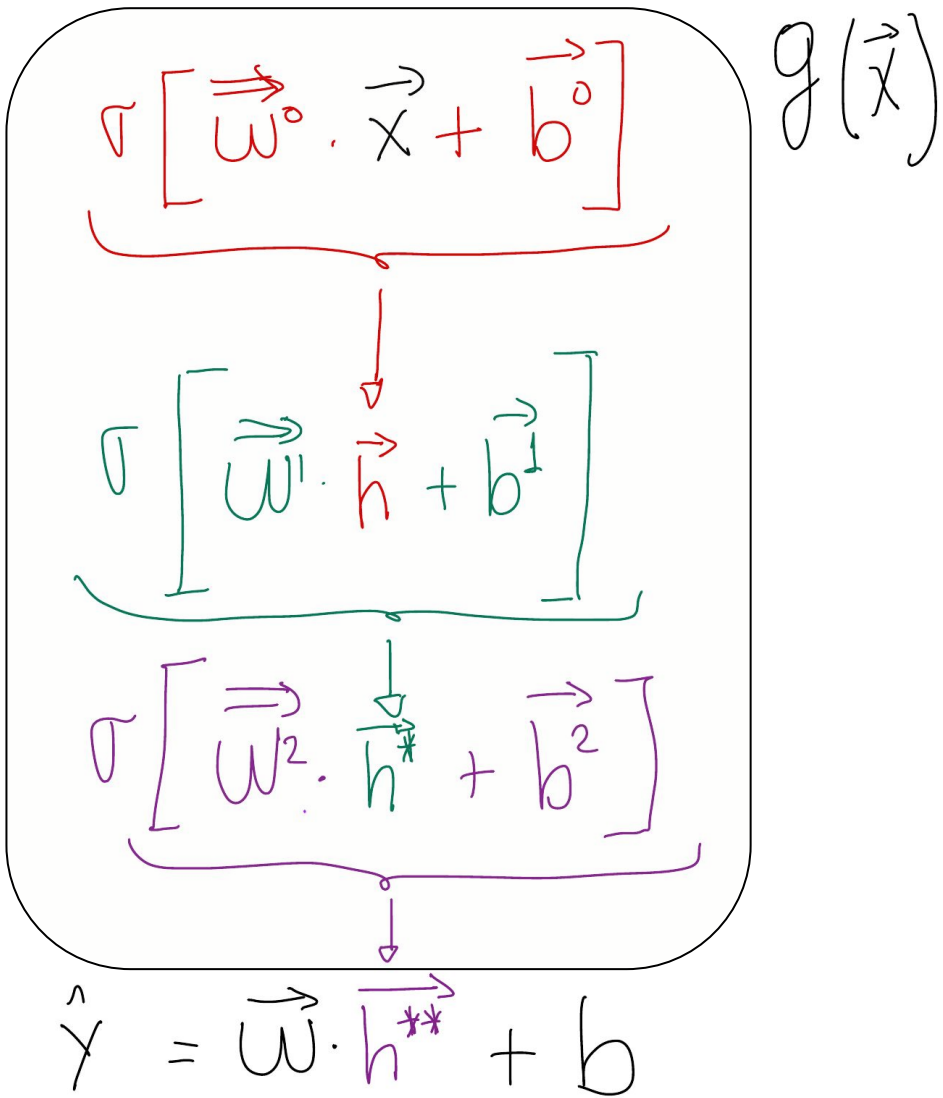
$$\underbrace{\sigma[\vec{w}^1 \cdot \vec{h} + b^1]}_{\downarrow}$$

$$\underbrace{\sigma[\vec{w}^2 \cdot \vec{h}^* + b^2]}_{\downarrow}$$

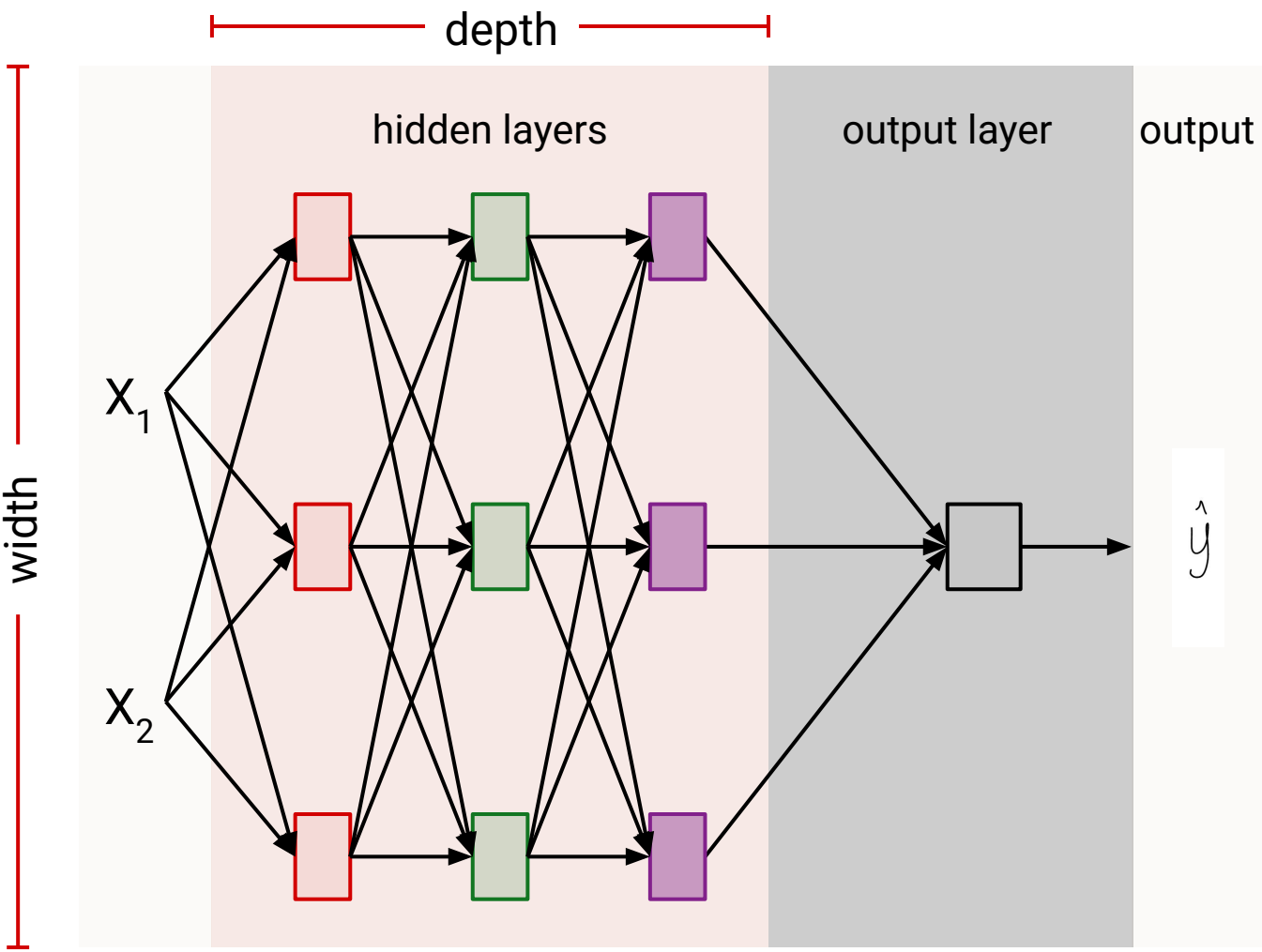
$$\hat{y} = \vec{w} \cdot \vec{h}^{**} + b$$



More “dense” layers



Number of free parameters



		2x3 example	one layer: D features F width	H layers: D features F width
1st hidden layer	weight matrix W^0	6	$F \cdot D$	$F \cdot D$
	bias b^0	3	F	F
other hidden layers	weights	–	–	$(H-1) \cdot F^2$
	biases	–	–	$(H-1) \cdot F$
output layer	weight matrix W	3	F	F
	bias b	1	1	1
Total		13	$F(D+2)+1$	?

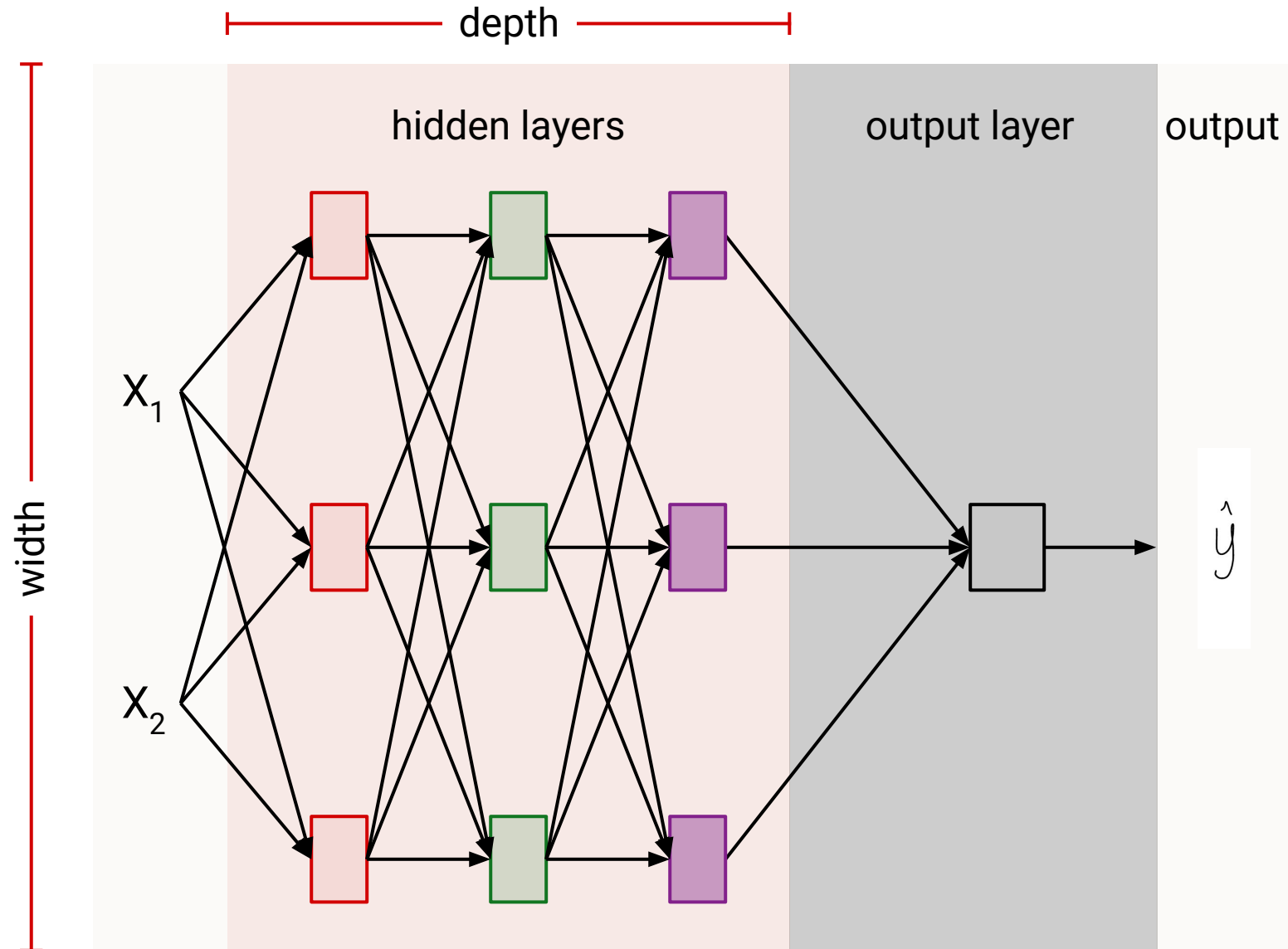
Why are neural networks a good choice?

Universal Approximation Theorem

a neural network can approximate any function $y=f(x)$, with a large enough network depth or width.

neural networks are, in the limit of large depth or width, expressive enough to fit any function.

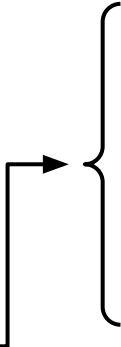
Andrew White, Deep Learning for Molecules and Materials

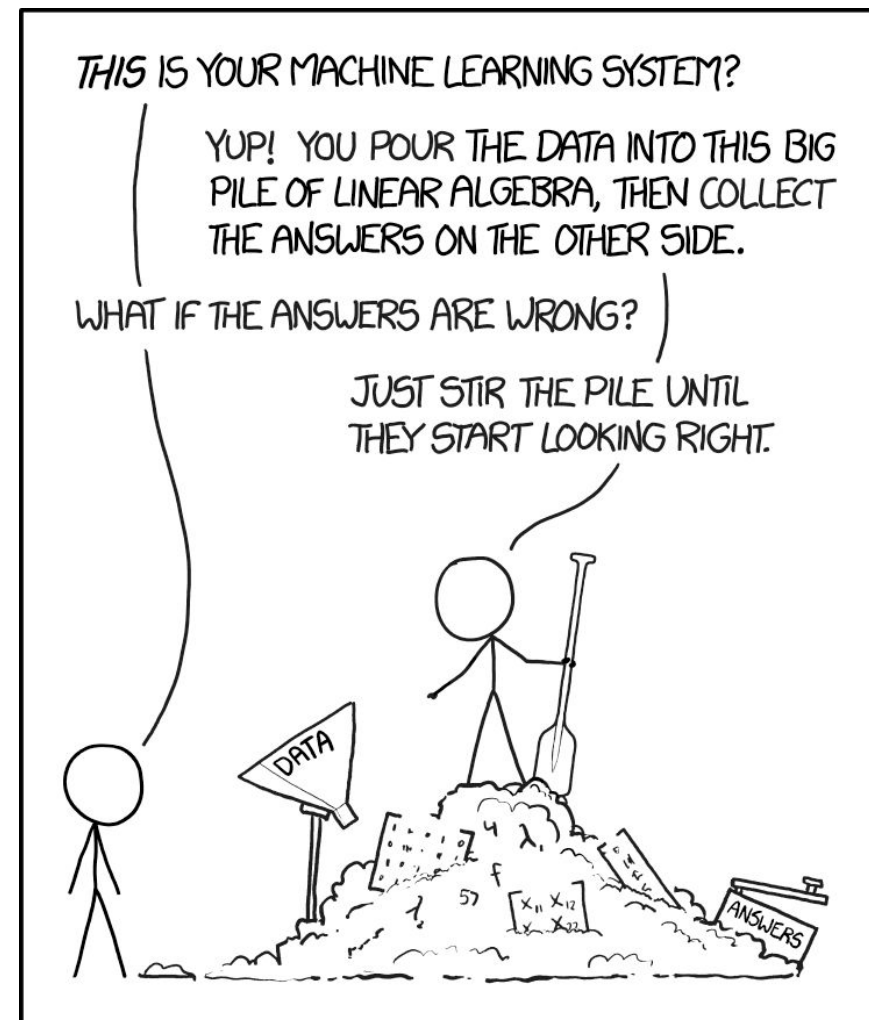


This was just an introduction! outlook list is long ;-):

- more complex layer architectures (convolutional)
- unsupervised learning (clustering & random forest)
- graph neural networks
- variational autoencoders
- ...
- Further reading: Andrew White, *Deep Learning for Molecules and Materials*. Online book.

Life-like data challenges:

- | | | |
|---|--|---|
| <ul style="list-style-type: none">• Generation• Curation & Maintenance• Storage• Use |  | <ul style="list-style-type: none">• extract relevant information out of the data• build models & simulations with predictive power |
|---|--|---|



xkcd.com/1838

Machine learning &
related-tools can help!

hands on

- Jupyter notebook:

<https://colab.research.google.com/drive/1dnUI71WlqvIfDWjJYPG0YKceQWlwSGkK?usp=sharing>

- Based on Andrew White, *Deep Learning for Molecules and Materials*. Online book.